KIỂM THỬ & ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

Bài 11- Thực hiện Quy trình Kiểm thử (2)
Báo cáo & Quản lý lỗi

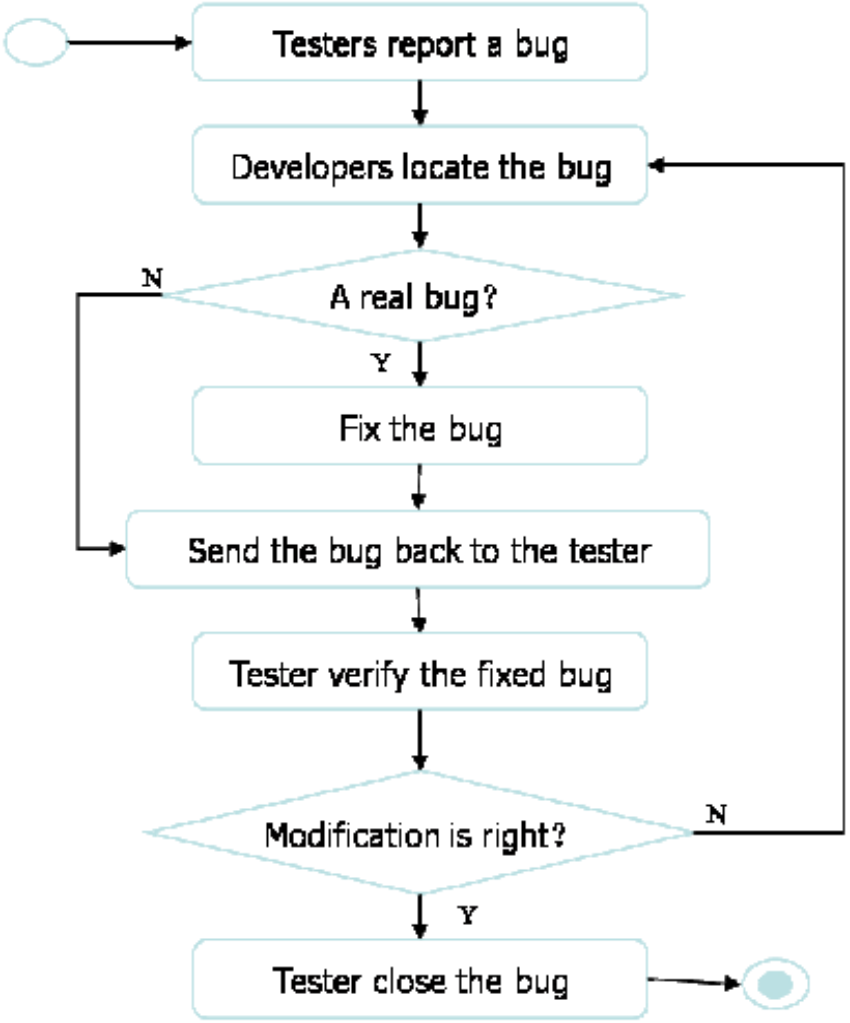
NGUYỄN THANH TRUNG
Sđt: 0935 885 687
email: thanhtrung05@gmail.com

NỘI DUNG

- Quản lý lỗi
- Báo cáo lỗi
- Báo cáo kết quả kiểm thử
- Đo lường hiệu quả kiểm thử

8.4. Quản lý lỗi

Quy trình theo dõi lỗi phần mềm



A simple bug Tracking process

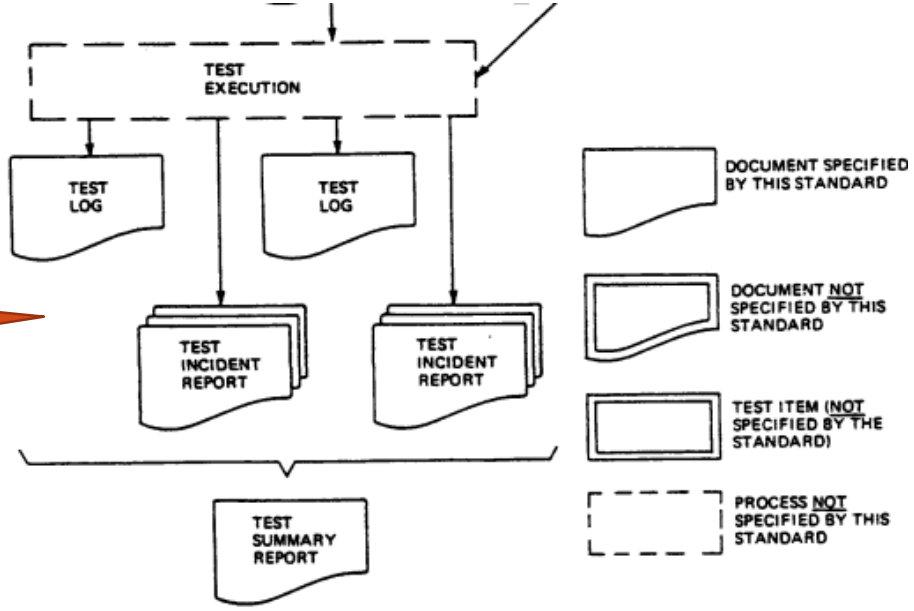
Mục tiêu

- Báo cáo kết quả kiểm thử
- Quản lý lỗi
  - Defect Tracking Processes
  - Defect Lifecycles
  - Defect Report Developments
  - Defect Management Systems
- Công cụ

1. Test Report (Báo cáo kết quả kiểm thử)

- Test report
- samples

Những tài liệu liên dùng báo cáo kết quả kiểm thử do IEEE đề xuất



Test Report

Test report là hoạt động ghi lại quá trình kiểm thử, kết quả của hoạt động kiểm thử,…

Một số tổ chức phát triển phần mềm thực hiện các loại báo cáo sau:

- Test log
- Test Result Report
- Test Release Report

Xem mẫu Test Report

TEST REPORT

Project Name	*Student Management*	**Creator**	*Ha Thi Thu Huong*
Project Code	*SM-2009*	**Reviewer/Approver**	*Van Kim Ngan*
Document Name	*SM-2009_Test Report_vx.x*	**Issue Date**	*<Date when this test report is created>*
Notes			

No	Module code	Pass	Fail	Untested	N/A	Number of test cases
1	Account Management	0	0	7	0	7
2	Subject Management	0	0	9	0	9
4	GUI	0	0	0	0	0
	Sub total	0	0	16	0	16

Test coverage **0.00** %
Test successful coverage **0.00** %

a. Test Log

- Test log: Là tài liệu ghi lại theo trật tự thời gian những sự kiện test đã được thực hiện và kết quả.
- Ai làm Test log?
  - Tester
- Khi nào làm Test log?
  - Trong khi thực hiện test.

b. Test Result Report

- Test result report là gì?
  - Là tài liệu báo cáo kết quả thực hiện test.
- Ai làm Test result report?
  - Tester
- Khi nào làm Test result report?
  - Sau khi thực hiện test xong một test item.
  - Sau khi test được một khoảng thời gian nào đó.
  - Sau khi test xong một lượt của toàn hệ thống hay một tiểu hệ thống.
  - Khi cần cung cấp thông tin cho người
  - quản lý dự án
  - Khi Test leader yêu cầu

TEST REPORT				
Project Name				
Type of Testing (U/I/F/A)				
Item Id			Coded by	
Test Case No	Test data (if any) / Test desc	Expected Result	Actual Result	
			1st Cycle	2nd Cycle
				Start Time:
				End Time:
Total Succeeded				
Failed				
Not Applicable				
Remarks				
1st Cycle	Tested By		Date	
2nd Cycle	Tested By		Date	
Recommended for release		Signature	Date	

Sample Detailed Test Report

b. Test Result Report (tiếp)

- Nội dung căn bản của Test result report
  - Giới thiệu tài liệu
  - Giới thiệu các test item mục tiêu
  - Các test case dự kiến thực hiện
  - Các test case đã thực hiện
  - Kết quả chi tiết của việc thực hiện từng test case
  - Thống kê tỉ lệ test case được thực hiện và tỉ lệ pass của các test case.
  - Đưa ra các change request
  - Ghi chú về các hiện tượng cần theo dõi thêm

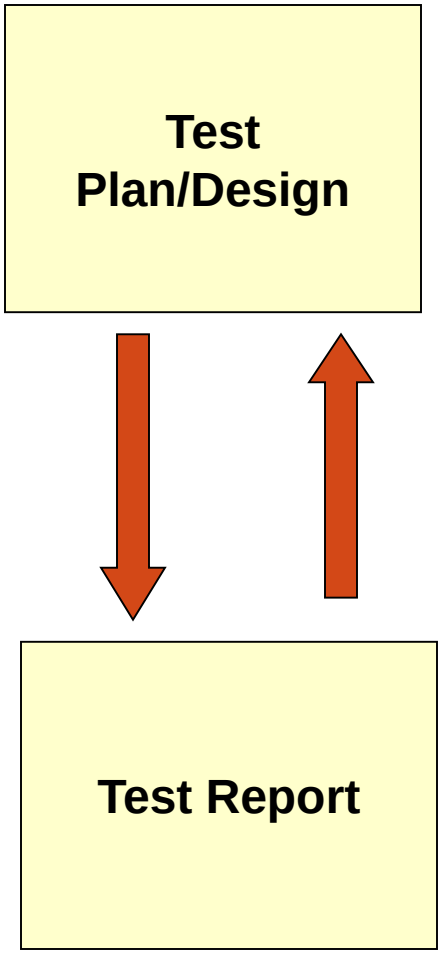
c. Test Release Report

- Test release report là gì?
  - Là tài liệu báo cáo kết quả test của sản phẩm, đánh giá chất lượng sản phẩm đã đủ tốt để release hay chưa?
- Ai làm Test release report?
  - Test leader
- Khi nào làm Test release report?
  - Khi ngừng test một bản sản phẩm
  - Khi cần kết luận sản phẩm có thể được release hay chưa?

9.3.3. Test Release Report (tiếp)

- Nội dung căn bản của Test Release Report
  - Giới thiệu tài liệu
  - Giới thiệu hệ thống được test
  - Khái quát về kết quả test:
    - Đánh giá chung về hệ thống được test, kết luận release hoặc không.
    - Đánh giá ảnh hưởng của môi trường kiểm tra
    - Các nhận xét khuyến nghị
  - Kết quả test chi tiết
  - Các phụ lục: thống kê số lỗi, số test case được thực hiện,…

Mối quan hệ giữa test report và các phần việc khác



- Xác định môi trường và các test cases sẽ thực hiện.
- Chỉ ra khối lượng test sẽ thực hiện.
- Giúp xác định được khi nào "test xong".

- Báo cáo kết quả test thực sự, đối chiếu với test plan.
- Đưa ra đánh giá, kết luận về độ tin cậy của chất lượng sản phẩm.
- Cho biết kế hoạch test đã được thực hiện trọn vẹn hay chưa.

2. Quản lý Defect

- Defect Management
- Quy trình theo dõi Defect
- Hệ thống quản lý Defect.

Tại sao cần quản lý lỗi phần mềm

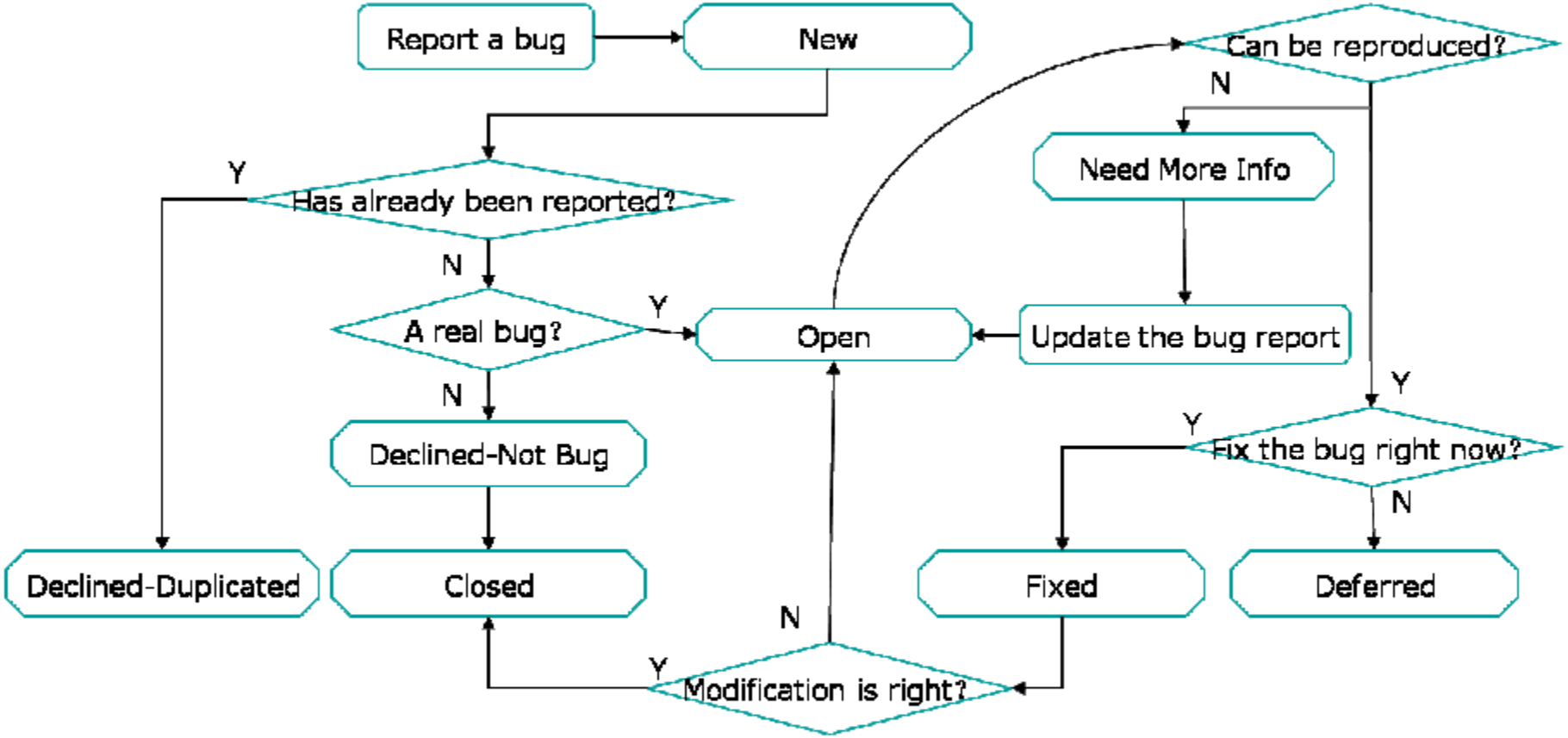
- Để đảm bảo các lỗi được tìm thấy trong suôt quá trình phát triển PM được báo cáo và xử lý.
- Để đảm bảo mỗi defect được ghi lại 1 cách hiệu quả, được kiểm tra và đóng lỗi (closed) một cách phù hợp, kịp thời.
- Phân tích cơ sở dữ liệu lỗi giúp xác định tình trạng hiện tại của dự án phần mềm và các vấn đề liên quan đến chất lượng sản phẩm.

Quy trình theo dõi Defect

Các dự án khác nhau có thể có quy trình quản lý lỗi khác nhau,

Tùy giai đoạn kiểm thử cũng có thể có quy trình quản lý lỗi khác nhau…

Defect Lifecycle



A simple Bug lifecycle

Các trạng thái lỗi

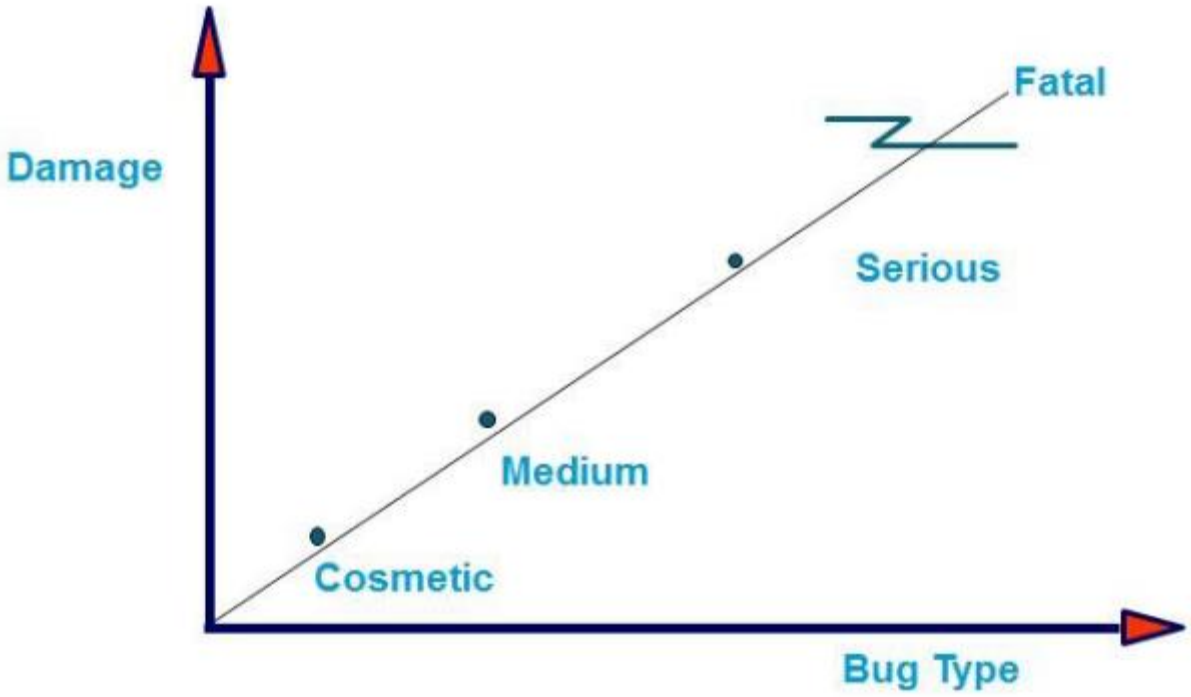
- Draft, New, Need more info,…
- Open: Lỗi đã được xác định và được sửa
- Fixed: Lỗi đã sửa
- Declined: Defect không xem là lỗi hoặc đã được báo cáo
- Closed: Lỗi sau khi sửa đã được tester xác nhận
- Deferred: Lỗi sẽ sửa sau…

The Structure of Defect Report

- Defect report structure
  - Different projects may have different defect report structures due to different defect tracking processes. But the basic structure information remains the same.
    - Defect identifier
    - Defect title
    - Basic information of the defect
      - Hardware and Software
      - Version
      - Type
      - Severity
      - Priority
    - Operation steps of reproduction
    - Attachment
  - Please note that the basic information is only a part of the whole defect report.

© 2009, CMU-ISR

Mức độ nghiêm trọng của lỗi



Cách viết Defect Report

- Defect reports là 1 trong những nhiệm vụ chính của Tester, viết báo cáo lỗi tốt góp phần cho việc quản lý, sửa lỗi và cải tiến quy trình sau này,….
- Vì vậy cần phải viết:

- Chính xác: Mô tả từng phần của bản báo cáo lỗi cho chính xác
- Rõ ràng: Mô tả rõ ràng giúp dễ đọc, dễ hiểu.
- Ngắn gọn với những thông tin chính nhưng đảm bảo những thông tin về lỗi.
- Nhất quán: Theo 1 định dạng chung,…

- Nói thêm về mô tả và phân loại lỗi

3. Defect Metrics

- Dựa vào các báo cáo về lỗi có thể trả lời những câu hỏi liên quan:
  - Module nào phát hiện nhiều lỗi nhất
  - Tester nào phát hiện nhiều lỗi nhất
  - Tỉ lệ các loại lỗi khác nhau
  - Dev có đúng tiến độ không
  - Tỉ lệ lối được sửa đạt yêu cầu trong lần đầu
  - Có bao nhiêu lỗi báo cáo trùng lắp
  - Phần mềm được phát hành đúng hạn không,…

3. Defect Metrics

- **Các thông số cần đo lường:**
  - Số lượng hoặc trung bình bao nhiêu lỗi mới được báo cáo mỗi ngày hoặc mỗi tuần
  - Số lượng hoặc trung bình bao nhiêu lỗi được sửa (fix) mỗi ngày hoặc mỗi tuần
  - Số lượng lỗi được mở mới/ đóng mỗi ngày hoặc mỗi tuần
  - Tổng số lỗi được báo cáo, được sửa lỗi
  - Tổng số mỗi loại lỗi theo mức độ nghiêm trọng của lỗi
  - Mối quan hệ giữa các module với các lỗi được tìm thấy
  - Số lỗi trên mỗi flatform, mỗi nhóm test, mỗi cá nhân tester
  - Thời gian sửa lỗi
  - Chỉ số DRE (Defect Removal Efficiency),…

Đo lường hiệu quả kiểm thử (Test effectiveness metrics)

Đánh giá hiệu quả test, chất lượng sản phẩm PM,…

Phân loại Test Effectiveness Metrics

- Đo lường sự hài lòng của khác hàng
- Đo lường các lỗi như: Số lỗi được phát hiện trong quá trình kiểm thử, số lỗi phát hiện bởi người sử dụng, độ nghiêm trọng của lỗi, thời gian tiềm ẩn của lỗi, mật độ và phân bố lỗi, …

Đánh giá số lỗi tìm thấy trong quá trình test

- Tất cả các lỗi không như nhau, có các mức độ tác động khác nhau, việc phát hiện lỗi góp phần nâng cao chất lượng PM.
- Ví dụ dùng bảng dự đoán và so sánh với kết quả thực tế, bảng này dự đoán đựa vào phiên bản trước đố hoạc các dựa án trước đó

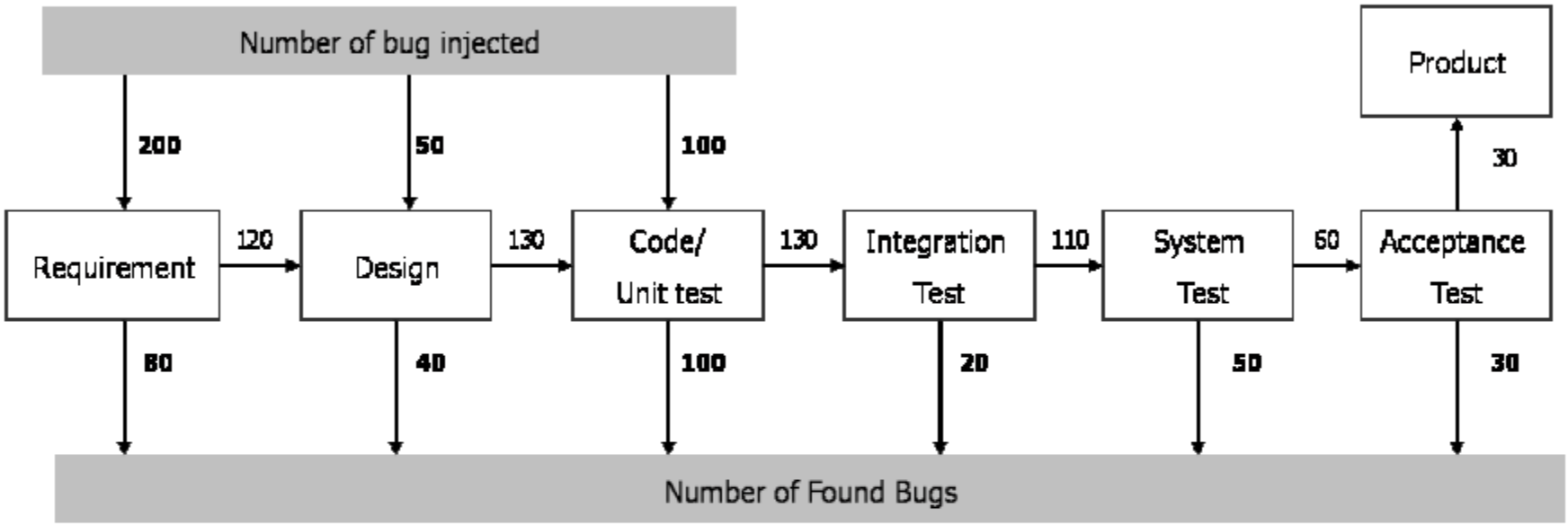
	Total Number Predicted	Predicted (P) Versus Actual(A)									
		Jan		Feb		Mar		Apr		May	
		P	A	P	A	P	A	P	A	P	A
Requirement Review	20	20	14								
Design Review	5	5	0	15		15					
Test Design	65			25		30		10			
Code Inspection	120							60		60	
Unit Test	80										
System Test	40										
Regression Test	10										
Acceptance Test	5										
6 Months Production	15										
Totals	390	25	14	40		45		70		60	

Defect Removal Efficiency (DRE)

DRE = Total number of defect found / Total number of defects

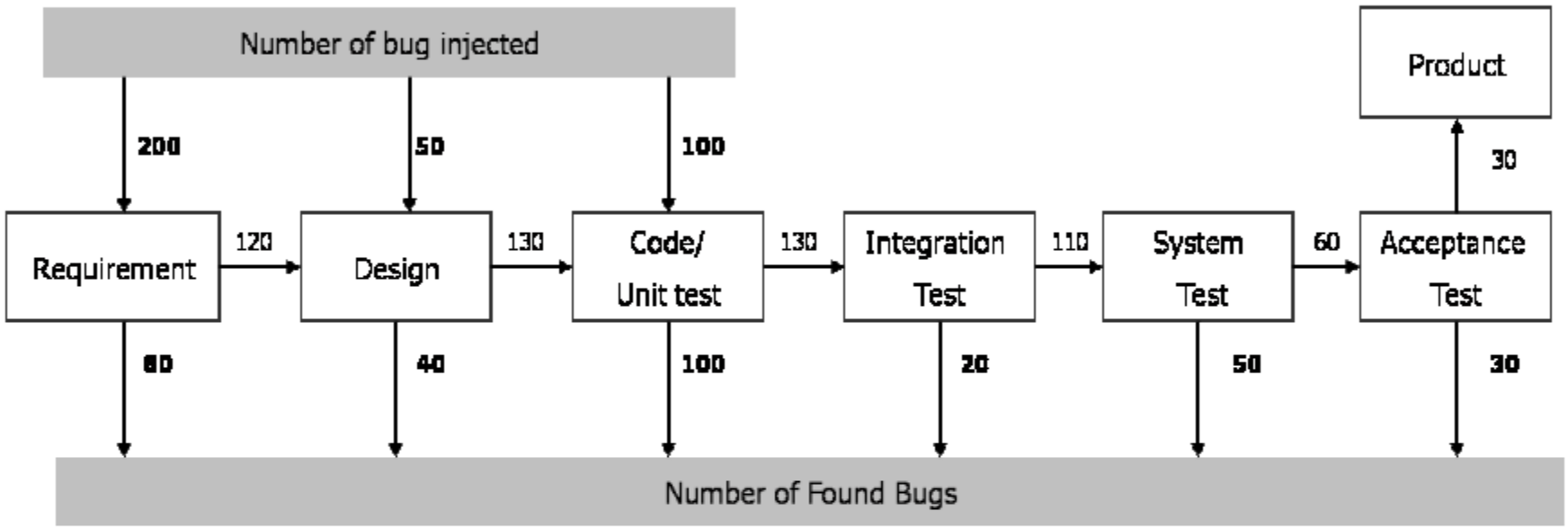
- Total number of defect found: Tổng số lỗi phát hiện trong quá trình phát triển
- Total number of defects: Tổng số lỗi thực tế được tìm thấy kể cả được người sử dụng phát hiện.

DRE Example



- Number of Bugs Found in Testing = 80+40+100+20+50+30=320
- Number Not Found= 30
- DRE = 320/(320+30) = 0.91

DRE Example



$$\text{System Test DRE} = \frac{\text{Number of Bugs Found in System Test}}{\text{Number of Bugs Found in System Test} + \text{Number of Bugs Found in Acceptance Test and Product}}$$

$$= \frac{50}{50+30+30} = 0.45$$

© 2009, CMU-ISR

Defect Age

- An effective testing effort would tend to find defects earlier than a less effective testing effort.
- You may adopt the table below to reflect your defect age. For example, a requirement defect discovered during a high-level design review would be assigned a defect age of 1. If this defect had not been found until the Pilot, it would have been assigned a defect age of 8.

Phase Created	Requirement	High Level Design	Low Level Design	Coding	Unit Testing	Integrating testing	System Testing	Acceptance Testing	Pilot	Product
Requirement	0	1	2	3	4	5	6	7	8	9
High-Level Design		0	1	2	3	4	5	6	7	8
Low Level design			0	1	2	3	4	5	6	7
Coding				0	1	2	3	4	5	6
Summary										

Defect Spoilage

- Formula

$$\textbf{Spoilage} = \frac{\Sigma(\textbf{Number of Bugs} * \textbf{Discovered Defect Age})}{\textbf{Total Number of Bugs}}$$

- Defect spoilage is a metric that uses defect age and distribution of defects to measure the effectiveness of defect removal activities.
- Lower values for spoilage indicate more effective defect discovery processes.
- Next slide shows an example of defect spoilage and the defect age is based on previous slide, for example:

$$\textbf{Coding Spoilage} = \frac{62*1+16*2+6*3+2*4+3*5+20*6}{62+16+6+2+3+20} = \frac{255}{109}$$

$$= 2.7$$

Defect Spoilage Example

Phase Created	Phase Discovered										Total Defects
	Requirement	High Level Design	Low Level Design	Coding	Unit Testing	Integrating testing	System Testing	Acceptance Testing	Pilot	Product	
Requirement	0	8	4	1	0	0	5	6	2	1	27
High-Level Design		0	9	3	0	1	3	1	2	1	20
Low Level design			0	15	3	4	0	0	1	8	31
Coding				0	62	16	6	2	3	20	109
Summary	0	8	13	19	65	21	14	9	8	30	187

Number of Defects found during

Phase Created	Phase Discovered										Defect Spoilage
	Requirement	High Level Design	Low Level Design	Coding	Unit Testing	Integrating testing	System Testing	Acceptance Testing	Pilot	Product	
Requirement	0	8	8	3	0	0	30	42	16	9	116/27 = 4.3
High-Level Design		0	9	6	0	4	15	6	14	8	62/20=2.1
Low Level design			0	15	6	12	0	0	6	56	95/31=3.1
Coding				0	62	32	18	8	15	120	255/109=2.7
Summary											514/187=2.7

Defect spoilage Based on defect age

© 2009, CMU-ISR

Defect Density

- Formula

Defect Density = Number of Defects / Lines of Code

- Defect density can help the tester focus on problematic parts of the system. Experience has shown that parts of a system where large quantities of defects have been discovered, will continue to have large numbers of defects, even after the initial cycle of testing and correcting of defects has taken place.
- By using this method, you should first answer two main questions:

- What is a defect and how to count?
- What is a line of code?

Mô tả và phân loại lỗi

- Làm gì ngay sau khi tìm thấy lỗi?
  - Lặp lại một lần nữa các thao tác dẫn đến bug
  - Tìm cách tổng quát hoá trường hợp dẫn đến bug.
    - Xét xem liệu còn những thao tác nào khác dẫn cũng dẫn bug?
    - Tìm ra các thao tác tối thiểu dẫn đến bug
    - Để ý xem dữ liệu test, cấu hình (settings) hay các option hiện tại có gì đặc biệt?
  - Xét xem liệu bug này có thể dẫn đến các hậu quả gì khác?
  - Tìm lướt xem bug này đã từng được ghi hay chưa?

44

Mô tả và phân loại lỗi (tiếp)

- Mô tả thật rõ ràng những bước tối thiểu để sinh lại bug.
- Không ôm đồm: mỗi bản ghi mô tả bug chỉ bao gồm một bug. Trong trường hợp tìm ra nhiều bug đồng thời, hãy tạo ra nhiều bản mô tả riêng biệt, ngay cả khi những bug đó có quan hệ với nhau. Điều này sẽ giúp việc nhận, xử lý và theo dõi bug được dễ dàng hơn.
- Không dùng nhiều từ tiếng Việt không dấu, không dùng các từ ngữ khó hiểu hoặc dễ gây nhầm lẫn.
- Đảm bảo rằng những người khác sẽ dễ dàng hiểu được bug.

45

Mô tả và phân loại lỗi (tiếp)

- Summary: Vị trí của nơi xảy ra bug trong phần mềm, kèm sau là test comment.
- Type: Kiểu lỗi, bao gồm:
  - Not set
  - Crash – data loss
  - Crash – not data loss
  - Incorrect functionality
  - Cosmetic
  - Feature request
  - Recommend

46

Mô tả và phân loại lỗi (tiếp)

- Theo mức độ ưu tiên (priority)
  - Not set
  - Critical
  - Hight
  - Medium
  - Low
  - Future release
  - Recomment

47

Theo mức độ ảnh hưởng (severity)
Theo dạng: High, medium và low, hoặc

- Program break
- Incomplete
- Unfriendly
- Cosmetic

Tóm tắt nội dung

Type	Priority	Severity
Crash (data loss + not data loss)	Critical	Program break
Incorrect functionality	Hight	Not expected/ Incomplete
Cosmetic	Hight/ Medium/ Low	Unfriendly/ Incomplete/ Interface
Feature request	Medium/ Low	incomplete

Bài tập Phân loại lỗi

49